

# Network Science

---

## Lecture 09: Node Representations and Graph Embeddings

Prof. Dr. Michael Granitzer

Dr. Kanishka Ghosh Dastidar

Dr. Jelena Mitrovic

# From Structure to Representation

Every previous lecture analysed the graph as a **combinatorial object**: edges, triangles, communities, cuts, shortest paths. Today we ask the dual question:

Can every node be represented as a *point* in  $\mathbb{R}^d$  such that graph structure is preserved as geometry?

If two nodes are "similar" in the graph (same community, similar role, co-visited by walks), they should be *close* as vectors.

# From Structure to Representation

Every previous lecture analysed the graph as a **combinatorial object**: edges, triangles, communities, cuts, shortest paths. Today we ask the dual question:

Can every node be represented as a *point* in  $\mathbb{R}^d$  such that graph structure is preserved as geometry?

If two nodes are "similar" in the graph (same community, similar role, co-visited by walks), they should be *close* as vectors.

This is **graph representation learning** — turning a discrete object into a continuous one so that we can use every tool from classical ML: clustering, classification, retrieval, generative models.

# The Question — Made Visual

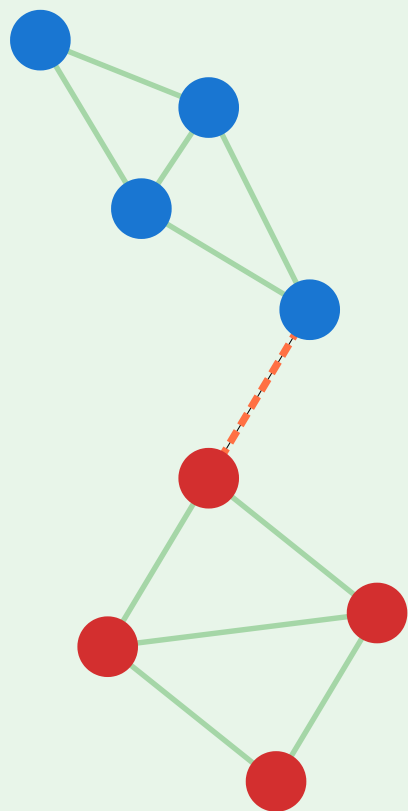


# Laplacian Eigenmaps — From Graph to Geometry

Eigenvectors of the Laplacian embed each node as a point in  $\mathbb{R}^d$

## 1. Graph G

two weakly connected clusters



colour = sign of Fiedler vector

## 2. Laplacian L

$$L = D - A$$

$$\begin{bmatrix} 2 & -1 & 0 & 0 & \dots \\ -1 & 3 & -1 & -1 & \dots \\ 0 & -1 & 3 & -1 & \dots \\ 0 & -1 & -1 & 2 & \dots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

### eigendecomposition

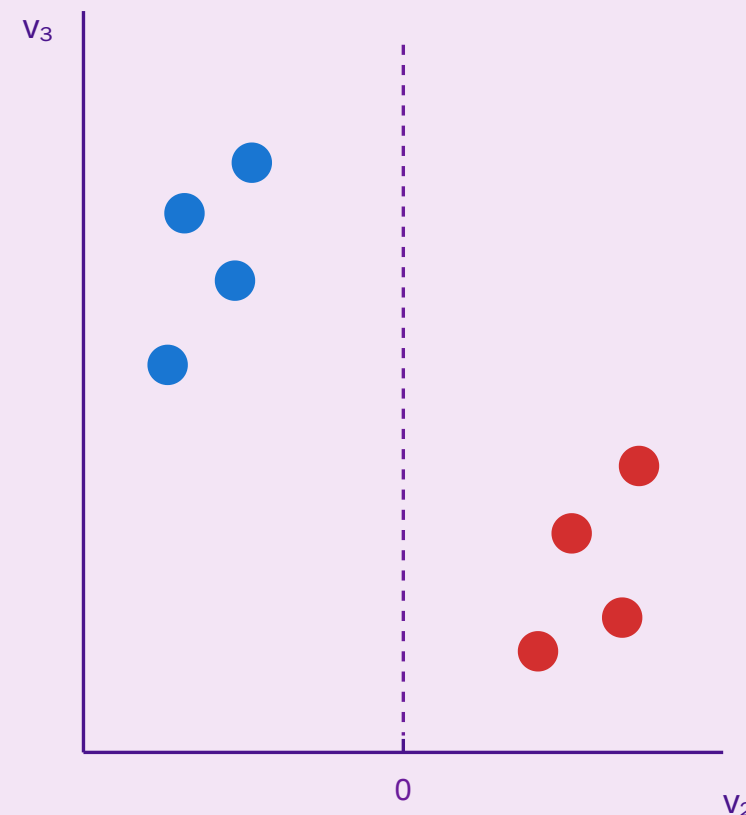
$$L v_k = \lambda_k v_k$$

$$0 = \lambda_1 \leq \lambda_2 \leq \lambda_3 \leq \dots$$

take  $v_2, v_3$  as coords

## 3. Embedding $\mathbb{R}^2$

$$z_v = (v_2(v), v_3(v))$$

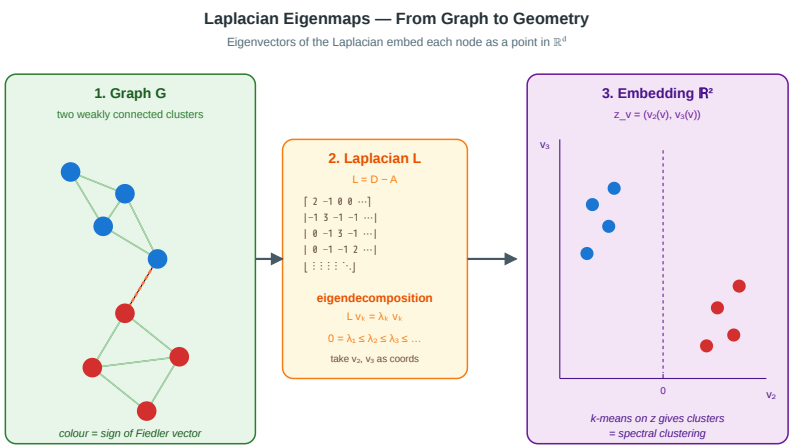


*k*-means on  $z$  gives clusters  
= spectral clustering

# A Sixth Gap — The Representation Gap

L07 (navigability) and L09 (representation) are the two faces of the same problem: making graphs *work* with local, vector-based operations.

| Lecture | Gap               | Strategy                      |
|---------|-------------------|-------------------------------|
| L07     | Navigational      | Multi-scale links             |
| L09     | Representation    | Embed nodes in $\mathbb{R}^d$ |
| L08     | Process-structure | Compare processes             |



# Takeaway

Graph representation learning turns a discrete object into a continuous one. The goal is a map  $z : V \rightarrow \mathbb{R}^d$  such that graph similarity  $\approx$  vector similarity. Different methods capture different notions of "similar."

# Spectral Foundations

**Guiding Question:** Can the eigenvectors of a graph matrix give us a natural geometry for its nodes?



# The Graph Laplacian — One More Time

For an undirected graph  $G = (V, E)$  with adjacency  $A$  and degree diagonal  $D$ :

$$L = D - A \quad (\text{combinatorial Laplacian})$$

$$L_{\text{sym}} = I - D^{-1/2} A D^{-1/2} \quad (\text{normalised Laplacian})$$

$L$  is symmetric positive semi-definite. Its eigenvalues satisfy  $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$ .

# The Graph Laplacian — One More Time

For an undirected graph  $G = (V, E)$  with adjacency  $A$  and degree diagonal  $D$ :

$$L = D - A \quad (\text{combinatorial Laplacian})$$

$$L_{\text{sym}} = I - D^{-1/2} A D^{-1/2} \quad (\text{normalised Laplacian})$$

$L$  is symmetric positive semi-definite. Its eigenvalues satisfy  $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$ .

The eigenvectors corresponding to the smallest non-zero eigenvalues are the most informative: they encode the graph's *large-scale* structure.

# Laplacian Eigenmaps (Belkin & Niyogi 2003)

Let  $v_2, v_3, \dots, v_{d+1}$  be the eigenvectors of  $L$  for the  $d$  smallest non-zero eigenvalues. Define

$$z_v = (v_2(v), v_3(v), \dots, v_{d+1}(v)) \in \mathbb{R}^d$$

Each node gets a  $d$ -dimensional embedding.

**Why this works.** The embedding is the solution to

$$\min_Z \sum_{(u,v) \in E} |z_u - z_v|^2 \quad \text{s.t. } Z^\top D Z = I$$

— connected nodes end up close, and the constraint prevents the trivial solution  $z = 0$ .

# Spectral Clustering = k-means on Laplacian Eigenmaps

Laplacian eigenmaps + k-means = **spectral clustering**.

- Nodes in the same community have similar low-frequency eigenvector values.
- $k$ -means on  $Z \in \mathbb{R}^{|V| \times d}$  recovers communities.
- This is how NCut (Shi & Malik 2000) and most image-segmentation algorithms work.

**Spectral gap.** The jump  $\lambda_{k+1} - \lambda_k$  indicates the "natural" number of communities. A large gap means the graph has *exactly*  $k$  clear clusters; a small gap means the partition is ambiguous.

# Signed Laplacian — Bridge to L06

For **signed networks** (L06, structural balance), the *signed Laplacian*  $\bar{L} = \bar{D} - A$  (with  $\bar{d}_i = \sum_j |A_{ij}|$ ) encodes enmity as well as friendship.

**Low eigenvectors of  $\bar{L}$**  embed nodes such that friends cluster and enemies separate — the continuous analogue of approximate structural balance.

# Signed Laplacian — Bridge to L06

For **signed networks** (L06, structural balance), the *signed Laplacian*  $\bar{L} = \bar{D} - A$  (with  $\bar{d}_i = \sum_j |A_{ij}|$ ) encodes enmity as well as friendship.

**Low eigenvectors of  $\bar{L}$**  embed nodes such that friends cluster and enemies separate — the continuous analogue of approximate structural balance.

Kunegis et al. (2010) used this to detect communities in signed online networks (Slashdot, Epinions) where positive and negative edges coexist.

# The Spectral Limitations

Why can't we just use spectral embeddings for everything?

1. **Cost.** Computing eigenvectors of a  $10^9 \times 10^9$  Laplacian is infeasible. Best sparse solvers still take  $O(|E| \cdot d)$  per iteration.
2. **Transductive.** A new node requires recomputing the whole spectrum — there is no "inference" step.
3. **No features.** Spectral embeddings use only graph structure; they ignore node attributes (text, images, tabular).
4. **Global.** All nodes see the same basis; local neighbourhood-specific patterns get averaged away.

# Takeaway

Spectral methods embed each node using the graph Laplacian's low eigenvectors. They are principled, globally optimal, and reduce clustering to linear algebra — but they scale poorly, are transductive, and ignore node features.



# Quick Check

You have a graph with 3 disconnected components of similar size.

1. What are the first three eigenvalues of the Laplacian  $L$ ?
2. What do the first three eigenvectors tell you?
3. What does "spectral gap" mean for this graph?

# Quick Check — Solution

$\lambda_1 = \lambda_2 = \lambda_3 = 0$  — the multiplicity of  $\lambda = 0$  equals the number of connected components.

They span the space of piecewise-constant vectors (constant on each component). Any indicator vector of one component is a linear combination of them.

The gap is between  $\lambda_3 = 0$  and  $\lambda_4$ . Large  $\lambda_4 \rightarrow$  components are well-separated and internally well-knit. Small  $\lambda_4 \rightarrow$  one component is close to splitting further.

# Quick Check — Solution

1.  $\lambda_1 = \lambda_2 = \lambda_3 = 0$  — the multiplicity of  $\lambda = 0$  equals the number of connected components.

They span the space of piecewise-constant vectors (constant on each component). Any indicator vector of one component is a linear combination of them.

The gap is between  $\lambda_3 = 0$  and  $\lambda_4$ . Large  $\lambda_4 \rightarrow$  components are well-separated and internally well-knit. Small  $\lambda_4 \rightarrow$  one component is close to splitting further.

# Quick Check — Solution

1.  $\lambda_1 = \lambda_2 = \lambda_3 = 0$  — the multiplicity of  $\lambda = 0$  equals the number of connected components.
2. They span the space of piecewise-constant vectors (constant on each component). Any indicator vector of one component is a linear combination of them.

The gap is between  $\lambda_3 = 0$  and  $\lambda_4$ . Large  $\lambda_4 \rightarrow$  components are well-separated and internally well-knit. Small  $\lambda_4 \rightarrow$  one component is close to splitting further.

# Quick Check — Solution

1.  $\lambda_1 = \lambda_2 = \lambda_3 = 0$  — the multiplicity of  $\lambda = 0$  equals the number of connected components.
2. They span the space of piecewise-constant vectors (constant on each component). Any indicator vector of one component is a linear combination of them.
3. The gap is between  $\lambda_3 = 0$  and  $\lambda_4$ . Large  $\lambda_4 \rightarrow$  components are well-separated and internally well-knit. Small  $\lambda_4 \rightarrow$  one component is close to splitting further.

# Random Walks as Representations — DeepWalk

**Guiding Question:** Can we use *samples* of graph structure (random walks) instead of spectral factorisation?

# The Analogy — Graph $\approx$ Corpus

| Language (word2vec) | Graph (DeepWalk)             |
|---------------------|------------------------------|
| Vocabulary          | Node set $V$                 |
| Corpus              | Collection of random walks   |
| Sentence            | One walk $v_1 v_2 \dots v_L$ |
| Word                | Node $v_i$                   |
| Context window      | Walk window of size $c$      |
| Skip-gram objective | Skip-gram objective          |

# The Analogy — Graph $\approx$ Corpus

| Language (word2vec) | Graph (DeepWalk)             |
|---------------------|------------------------------|
| Vocabulary          | Node set $V$                 |
| Corpus              | Collection of random walks   |
| Sentence            | One walk $v_1 v_2 \dots v_L$ |
| Word                | Node $v_i$                   |
| Context window      | Walk window of size $c$      |
| Skip-gram objective | Skip-gram objective          |

**Insight.** If we can generate many random walks from each node, we can reuse the entire word2vec machinery to learn node embeddings.



# DeepWalk Pipeline

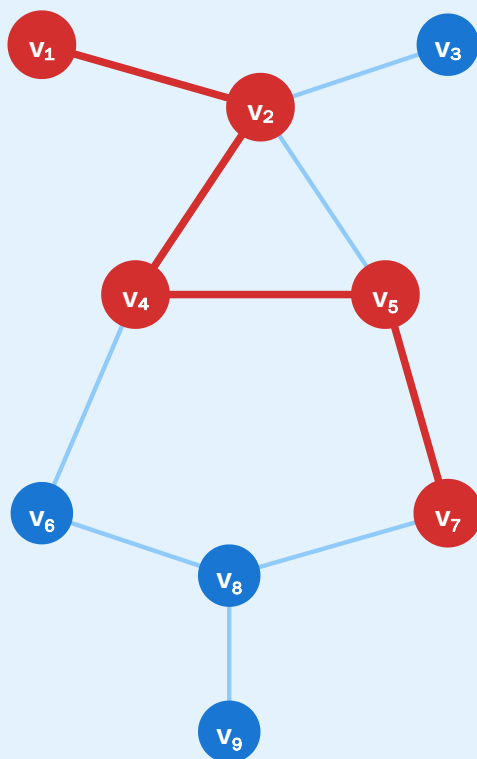


# DeepWalk — Random Walks as "Sentences" for Skip-Gram

node  $\approx$  word · walk  $\approx$  sentence · co-occurrence  $\rightarrow$  embedding

## 1. Random walks

length  $L$  from each node



walk:  $v_1 v_2 v_4 v_5 v_7$   
repeat  $y$  times per node

## 2. Skip-gram window

context  $c = 2$  around centre



centre  $v_4$  · context  $\{v_1, v_2, v_5, v_7\}$

training pairs  $(v, \text{context})$

|              |              |
|--------------|--------------|
| $(v_4, v_1)$ | $(v_4, v_2)$ |
| $(v_4, v_5)$ | $(v_4, v_7)$ |

### Skip-gram objective

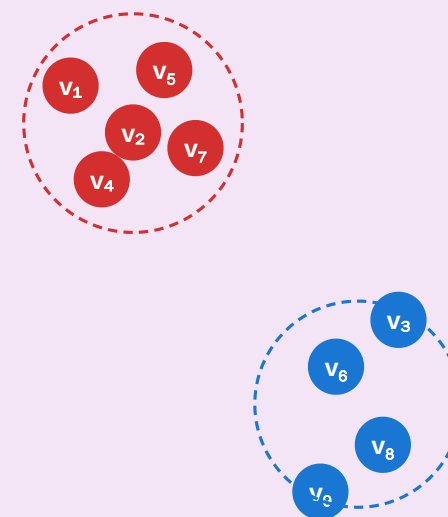
$$\max \sum \log P(u \mid v)$$

$$P(u \mid v) = \text{softmax}(z_u \cdot z_v)$$

"co-walking nodes have similar  $z$ "

## 3. Embeddings

$z_v \in \mathbb{R}^d, d \ll |V|$



co-walked nodes end up  
close in the embedding space

Perozzi et al. 2014

# The Skip-Gram Objective

Given walks  $W_i$  and window  $c$ , learn vectors  $z_v \in \mathbb{R}^d$  maximising

$$\mathcal{L} = \sum_i \sum_t \sum_{-c \leq j \leq c, j \neq 0} \log P(v_{t+j} \mid v_t)$$

with

$$P(u \mid v) = \frac{\exp(z_u \cdot z_v)}{\sum_{u' \in V} \exp(z_{u'} \cdot z_v)}$$

Dense softmax over  $|V|$  is too expensive  $\rightarrow$  **negative sampling** approximates it by  $k$  random non-context nodes per pair.

# What DeepWalk Captures

**Proposition (Qiu et al. 2018).** DeepWalk with long walks is *implicitly factorising* the log of a shifted PPMI matrix built from the random-walk co-occurrence matrix.

This connects DeepWalk back to spectral methods: both minimise a matrix-factorisation objective — the matrices just differ.

**Key result.** Random-walk methods and spectral methods sit on the same theoretical spectrum. DeepWalk is an approximate, sample-based spectral embedding of the walk co-occurrence matrix.

# Takeaway

DeepWalk treats a random walk as a sentence and reuses word2vec's skip-gram. It is scalable (SGD, no eigendecomposition) and unsupervised. Under the hood, it is factorising a co-occurrence matrix — the same machinery as spectral embedding, in a sampled form.

# node2vec — Biased Walks

**Guiding Question:** Should "similar" mean *same community* or *same structural role*? Can one embedding capture both?

# Two Notions of Similarity

**Homophily (community).** Two nodes are similar if they are in the same densely connected region — even if structurally different (one is a hub, the other is a leaf).

DFS-like walks wander far into one community.

**Structural equivalence (role).** Two nodes are similar if they play the *same role* — both are hubs, both are bridges — even if in different communities.

BFS-like walks stay local and sample neighbourhoods.

# Two Notions of Similarity

**Homophily (community).** Two nodes are similar if they are in the same densely connected region — even if structurally different (one is a hub, the other is a leaf).

DFS-like walks wander far into one community.

**Structural equivalence (role).** Two nodes are similar if they play the *same role* — both are hubs, both are bridges — even if in different communities.

BFS-like walks stay local and sample neighbourhoods.

DeepWalk's uniform walks give a blur between the two. node2vec lets us choose.



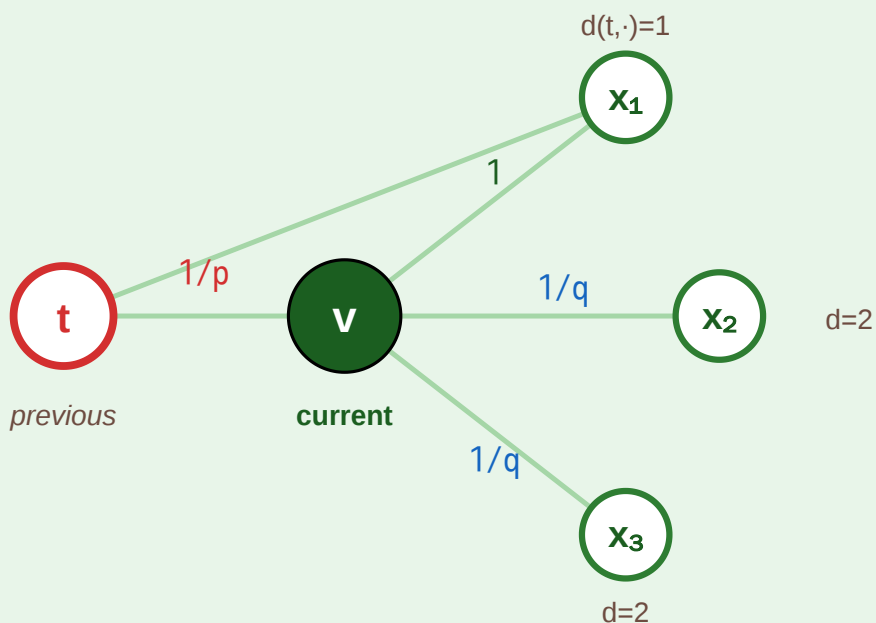
# The Biased Walk

# node2vec — Biased Random Walks with Return (p) and In-Out (q)

tune the walk between BFS-like (structural) and DFS-like (community) exploration

## 1. Second-order transition

biased by where we came from (t)



$\alpha(t \rightarrow v \rightarrow \cdot)$  based on distance  $d(t, \cdot)$ :

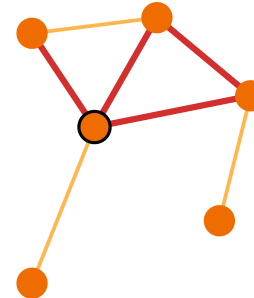
$d=0$  (back to  $t$ ):  $1/p$  ·  $d=1$ :  $1$  ·  $d=2$ :  $1/q$

## 2. Two regimes, two embeddings

$p, q$  shape what "similar" means

small  $q \rightarrow$  BFS-like

stay local, explore neighbours



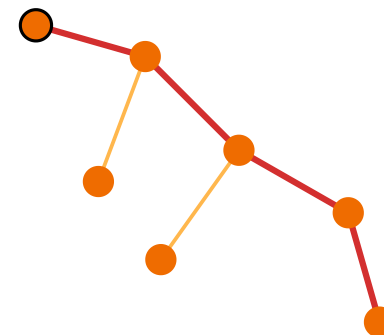
**captures structural roles**

hubs, bridges  $\rightarrow$  similar  $z$

"homophily by role"

small  $p \rightarrow$  DFS-like

wander far, reveal communities



**captures communities**

same cluster  $\rightarrow$  similar  $z$

Grover & Leskovec 2016 — one knob, two inductive biases

# node2vec in Production

Despite its age, node2vec is still widely deployed because:

- **Scalable.** Runs on billions of edges with sampled walks + SGD.
- **No features required.** Works on raw adjacency, making it a sensible *baseline* for any graph task.
- **Interpretable knobs.**  $p$  and  $q$  correspond to clear exploration policies.

# node2vec in Production

Despite its age, node2vec is still widely deployed because:

- **Scalable.** Runs on billions of edges with sampled walks + SGD.
- **No features required.** Works on raw adjacency, making it a sensible *baseline* for any graph task.
- **Interpretable knobs.**  $p$  and  $q$  correspond to clear exploration policies.

**Limitation.** Like DeepWalk, node2vec is **transductive** and **feature-free**. A new node requires a new walk + retraining; node attributes (text, numeric features) are ignored. These shortcomings motivate GNNs.

# Graph Neural Networks — Message Passing

**Guiding Question:** Can we learn node embeddings that *also* use node features — end-to-end with any downstream task?

# The Message-Passing Framework

At each layer  $l$ , every node  $v$  updates its representation by aggregating messages from its neighbours  $\mathcal{N}(v)$ :

$$h_v^{(l+1)} = \text{UPDATE}\left(h_v^{(l)}, \text{AGG}\left(h_u^{(l)} : u \in \mathcal{N}(v)\right)\right)$$

- AGG is a permutation-invariant function: sum, mean, max, or attention.
- UPDATE is usually a learnable linear transform + nonlinearity:  $\sigma(W[h_v, m_v])$ .
- Initial features  $h_v^{(0)}$  can be node attributes, one-hot IDs, or pretrained embeddings.

# The Message-Passing Framework

At each layer  $l$ , every node  $v$  updates its representation by aggregating messages from its neighbours  $\mathcal{N}(v)$ :

$$h_v^{(l+1)} = \text{UPDATE}\left(h_v^{(l)}, \text{AGG}\left(h_u^{(l)} : u \in \mathcal{N}(v)\right)\right)$$

- AGG is a permutation-invariant function: sum, mean, max, or attention.
- UPDATE is usually a learnable linear transform + nonlinearity:  $\sigma(W[h_v, m_v])$ .
- Initial features  $h_v^{(0)}$  can be node attributes, one-hot IDs, or pretrained embeddings.

Stacking  $L$  layers gives each node a *receptive field* of  $L$  hops — its final embedding reflects its  $L$ -hop neighbourhood.

# Message Passing — Visual



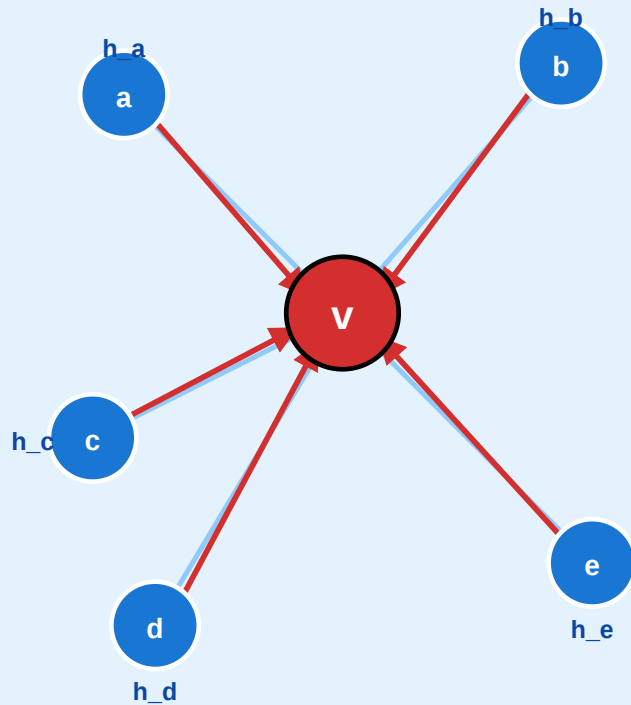


# Graph Neural Networks — Message Passing

$$h^{(l+1)}_v = \text{UPDATE}(h^{(l)}_v, \text{AGG}(\{h^{(l)}_u : u \in N(v)\}))$$

## 1. Neighbourhood of v

each node has a feature vector  $h_v$

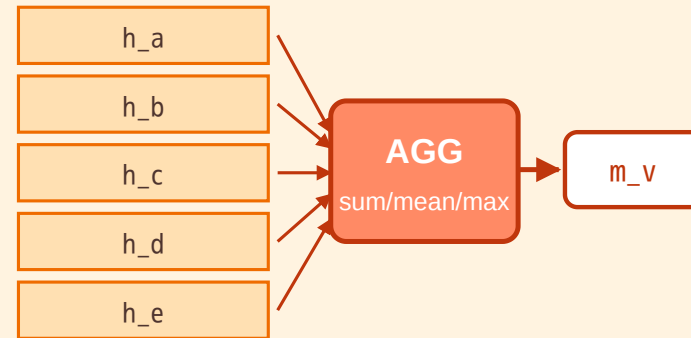


messages travel along edges

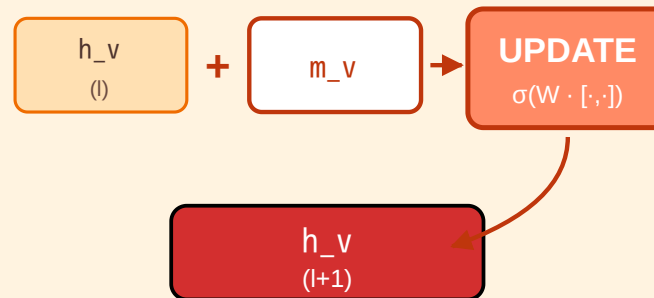
$$N(v) = \{a, b, c, d, e\}$$

## 2. Aggregate · Update

permutation-invariant pooling



then combine with own state



one GNN layer = one hop further

## 3. Stack L layers

receptive field = L hops

Layer 1 · 1-hop

Layer 2 · 2-hop

Layer 3 · 3-hop

### Common variants

- GCN — normalised sum
- GraphSAGE — sampling
- GAT — attention weights
- GIN — sum + MLP

too many layers →

**over-smoothing**

all nodes collapse to same z

# Four GNN Variants in One Line Each

- **GCN** (Kipf & Welling 2017) — AGG = normalised sum:  $\tilde{A} = D^{-1/2}(A + I)D^{-1/2}$ ; UPDATE =  $\sigma(\tilde{A}HW)$ .
- **GraphSAGE** (Hamilton et al. 2017) — AGG on a *sampled* subset of neighbours; scales to very large graphs; inductive by design.
- **GAT** (Veličković et al. 2018) — AGG weighted by learned *attention*  $\alpha_{uv}$ .
- **GIN** (Xu et al. 2019) — AGG = sum; UPDATE = MLP. Provably as powerful as the Weisfeiler-Lehman graph isomorphism test.

# The Over-Smoothing Problem

Stacking too many GNN layers kills the embedding.

- After  $L$  layers, each node's embedding averages over its  $L$ -hop neighbourhood.
- For random graphs, all nodes eventually see the entire graph  $\rightarrow$  all embeddings collapse to the same vector.
- This happens already at  $L = 3-5$  for typical benchmarks.

# The Over-Smoothing Problem

Stacking too many GNN layers kills the embedding.

- After  $L$  layers, each node's embedding averages over its  $L$ -hop neighbourhood.
- For random graphs, all nodes eventually see the entire graph  $\rightarrow$  all embeddings collapse to the same vector.
- This happens already at  $L = 3-5$  for typical benchmarks.

**Remedies.** Residual connections (JKNet), edge-weight normalisation, explicit node-pair features, or simply using fewer layers (1–3 is often enough).

# Takeaway

GNNs learn node embeddings by repeated message passing — each layer extends the receptive field by one hop. They are inductive, feature-aware, and end-to-end trainable. But they suffer from over-smoothing: more layers is not always better.

# Quick Check

You train a 2-layer GCN on a citation network to predict paper topics.

1. What is the receptive field of each paper's final embedding?
2. Why would stacking 10 layers hurt performance?
3. If you add a new paper tomorrow, can you predict its topic without retraining?

# Closing the Loop — Embeddings Meet HNSW

**Guiding Question:** How do the embeddings we just built power modern retrieval systems — and what does this have to do with Kleinberg's grid?

# The Full Pipeline

1. **Build embeddings** — spectral, DeepWalk, node2vec, or a trained GNN produces  $z_v \in \mathbb{R}^d$  for each node.
2. **Build an index** — HNSW (L07) or similar ANN structure over  $z_v : v \in V$ .
3. **Query** — encode query  $q$  as  $z_q$ , return top- $k$  nearest  $z_v$  in  $O(\log n)$ .
4. **Use** — link prediction, recommendation, RAG retrieval, semantic search.



# The Full Pipeline

1. **Build embeddings** — spectral, DeepWalk, node2vec, or a trained GNN produces  $z_v \in \mathbb{R}^d$  for each node.
2. **Build an index** — HNSW (L07) or similar ANN structure over  $z_v : v \in V$ .
3. **Query** — encode query  $q$  as  $z_q$ , return top- $k$  nearest  $z_v$  in  $O(\log n)$ .
4. **Use** — link prediction, recommendation, RAG retrieval, semantic search.

The same pipeline powers **product recommendation** (Alibaba, Amazon), **friend suggestion** (LinkedIn, Meta), **fraud detection** (PayPal), and **retrieval-augmented LLMs** (2024–2026 production stacks).

# Why Geometry Matters

HNSW needs the embedding space to have the **right geometry** for greedy descent to work:

- **Similar nodes close, dissimilar nodes far** — true by construction in spectral / DeepWalk / GNN embeddings.
- **Low intrinsic dimension** — if embeddings live on a low-dimensional manifold inside  $\mathbb{R}^d$ , greedy search is efficient. Otherwise we hit the curse of dimensionality.
- **Smoothness** — continuous similarity (not sharp drops), which is why normalisation, temperature, and contrastive losses matter.

The *quality* of the embedding determines whether the index is actually navigable.

# Takeaway

Embeddings (L09) and navigable indexes (L07) are the two halves of the modern retrieval pipeline. Spectral, DeepWalk, node2vec, and GNN methods produce the vectors; HNSW descends them. Kleinberg's insight ( $r = d$ ) reappears as an engineering guarantee in HNSW's layer structure.

# Wrap-Up — The Representation Arc

| Method                | Uses                         | Scales to         | Inductive | Feature-aware |
|-----------------------|------------------------------|-------------------|-----------|---------------|
| Laplacian Eigenmaps   | eigendecomposition           | $\leq 10^6$ nodes | ✗         | ✗             |
| DeepWalk / node2vec   | random walks + skip-gram     | $10^9$ edges      | ✗         | ✗             |
| GCN / GraphSAGE / GAT | message passing              | $10^9$ edges      | ✓         | ✓             |
| Combined w/ HNSW      | any of the above → ANN index | $10^9$ vectors    | ✓         | ✓             |

Each row *adds* a capability without replacing the previous row — classical spectral embeddings remain the theoretical benchmark, random-walk methods remain the unsupervised baseline, GNNs add features, HNSW adds retrieval.

## Looking Ahead: Exercise and Beyond

The exercise for this lecture compares Laplacian Eigenmaps, node2vec, and a small GCN on a shared dataset. Real graph-ML research in 2024–2026 has moved on to **graph transformers**, **graph foundation models**, and **heterogeneous GNNs** — but all rest on the message-passing core you now understand.

# Reading

Chapter 23 of Hamilton's *Graph Representation Learning* (2020) covers spectral, random-walk, and GNN embeddings. Perozzi et al. (2014) is DeepWalk. Grover & Leskovec (2016) is node2vec. Kipf & Welling (2017) introduces GCN. Hamilton et al. (2017) introduces GraphSAGE. Malkov & Yashunin (2018) is the HNSW paper that closes the loop to L07.